

420-126-LI

POO 1

Introduction à la programmation

Plan

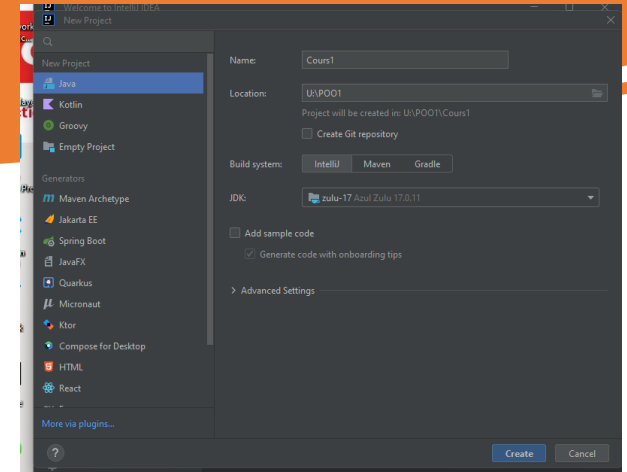
- Premier programme
- Compiler-exécuter
- Instructions Java

- Lignes blanches et commentaires
- Affichage à la console
- Opération sur les nombres
- **Découverte de votre IDE**

Premier programme Java (1)

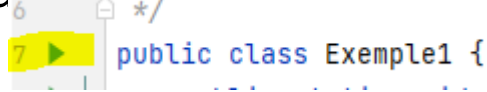
- Ouvrir IntelliJ (depuis le finder , recherche Windows)
- Créer un nouveau projet (bouton new project),
- Choisir un nom, un emplacement de stockage et décochez le « Add sample code »
- Appuyez sur « Create »
- Dans « File/New » choisissez « Java class » et nommez-la « Exemple1 », cela crée un fichier « Exemple1.java » qui contient un **code java**
- Modifiez le code comme ceci:

```
public class Exemple1 {  
  
    public static void main(String[] args) {  
        int annee = 2026;  
        System.out.println("Voici un programme java de " + annee);  
    }  
}
```



Premier programme Java (2)

- Exécuter le programme en cliquant sur la flèche verte.



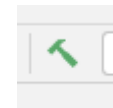
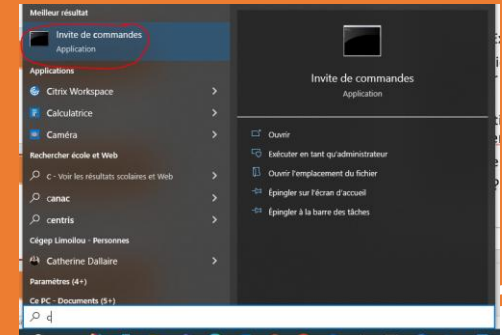
- Le programme « s'exécute » et le résultat s'affiche dans la console:



- On peut modifier un peu le code pour essayer de comprendre le lien entre code et résultat.
- Essayons maintenant de décortiquer ce qui se passe derrière cet appui sur le bouton vert...

Compiler-Exécuter

- Copier le fichier Exemple1.java dans
C:\Users\nom.prenom\Documents
- Ouvrir une « Invite de commande » (Finder cmd)
- Taper `cd C:\Users\votreDA\Documents`
- Taper `javac Exemple1.java` (Vous venez de **compiler** votre premier **fichier** java)
- Taper `java Exemple1` (Vous venez **d'exécuter** votre premier **programme** java)
- La **compilation** fabrique un fichier binaire (.class, « bytecode ») qui peut être maintenant compris par la machine (en fait c'est la JVM, Java Virtual Machine), et donc **exécuté**.
- Il est fondamental de garder à l'esprit que ces deux étapes existent et ont des objectifs et contraintes différentes.
- Dans la pratique on utilisera notre **IDE** (« Integrated Development Environment ») pour se simplifier beaucoup la vie (ici **IntelliJ**)
- Remarquons le **petit marteau** qui représente la compilation,
- Cette étape précède automatiquement l'exécution



Instruction Java

- Une instruction est une opération simple que doit exécuter le programme.
- En java les instructions sont **séparées par des « ; »**.
- On regroupe les instructions dans des fichier texte d'extension «.java» qui forment le **code source** de notre **programme**
- Dans notre exemple on a **deux instructions**, le reste est du code qui permet de structurer le programme que l'on étudiera bientôt:

```
public class Exemple1 {  
  
    public static void main(String[] args) {  
        int annee = 2026;  
        System.out.println("Voici un programme java de " + annee);  
    }  
}
```

Lignes blanches et commentaires

- Une **ligne blanche** n'est pas une instruction et est ignorée lors de la compilation. Le Java permet de mettre autant de lignes blanches que l'on veut. Elles permettent de séparer des blocs de code.
- Un **commentaire** n'est pas une instruction. Il permet d'insérer du texte qui ne sera pas compilé. Il donne la possibilité au programmeur d'ajouter du texte au code source. **De documenter le code.**
- Les commentaires peuvent prendre deux formes:
 - 1) **//** cette ligne est un commentaire
 - 2) **/*** tout ce qui est entre deux est un commentaire
Et peut être sur plusieurs lignes!! ***/**

Lignes blanches et commentaires (suite)

- Exemple 2 (voir Exemple2.java)

```
3 ▶ public class Exemple2 {  
4 ▶     public static void main(String[] args) {  
5  
6         // Un commentaire pour donner l'exemple  
7         int nbArticle = 10; // il y a 10 articles  
8  
9         System.out.print("Vous avez " + nbArticle + " articles en stock.");  
10  
11         // Ici, on veut simplement ignorer temporairement une instruction  
12         // Celle-ci ne sera pas exécutée.  
13  
14         //System.out.print("Au revoir!");  
15  
16         /* Autre forme de commentaire  
17            sur plusieurs lignes  
18         */  
19     }  
20 }
```

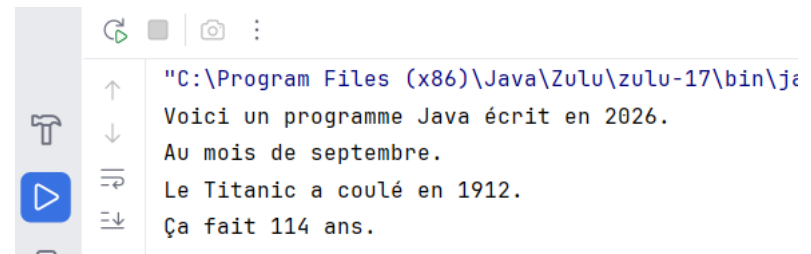
- Nous verrons que les commentaires permettront au programmeur de documenter son code.
- Il donne aussi la possibilité au programmeur de **neutraliser** (ne pas exécuter) temporairement une ligne de code.

Affichage à la console

- On peut utiliser les **méthodes** « `print` » et « `println` » de la **l'objet** « `System.out` » pour afficher un texte (appelé chaîne de caractère) ou le résultat d'un calcul mathématique (expression).
- La différence entre le « `System.out.println` » et le « `System.out.print` » est que le premier **ajoute un saut de ligne** après l'affichage de la ligne.

Exemple 3 (voir Exemple3.java)

```
public class Exemple3 {  
    public static void main(String[] args) {  
        int annee = 2026;  
        int anneeTitanic = 1912;  
        System.out.print("Voici un programme Java écrit en ");  
        System.out.println(annee + ".");  
        System.out.println("Au mois de septembre.");  
        System.out.println("Le Titanic a coulé en 1912.");  
        System.out.print("Ça fait ");  
        System.out.print(annee - anneeTitanic);  
        System.out.println(" ans.");  
    }  
}
```



Opération sur des nombres (variables numériques)

- Prenons « $1 + 4$ ». Cette opération est une addition : « **la valeur de l'expression est** » le résultat de l'addition de 1 avec 4 et cette expression vaut 5.
- De la même manière, on peut utiliser toutes les opérations arithmétiques simples :
« $+ - * /$ »

- **Exemple**

```
4 + 1 //L'expression vaut 5
6 * 3 //L'expression vaut 18
9 / 3 //L'expression vaut 3
```

- **Notez bien** : les instructions précédentes sont inutiles parce qu'elles n'utilisent pas le résultat obtenu.
 - Une façon de l'utiliser serait d'en faire l'affichage.
 - Une autre serait de la stocker le résultat en mémoire, on en reparle...

Opération sur des nombres (variables numériques-suite)

- Exemple 4 (voir Exemple4.java)

```
public class Exemple4 {  
    public static void main(String[] args) {  
        System.out.println(1 + 1); //Affiche 2  
        System.out.println(4 + 1); //Affiche 5  
        System.out.println(6 * 3); //Affiche 18  
        System.out.println(9 / 3); //Affiche 3  
  
        //Avec des parenthese pour les problemes de priorite des operations  
        System.out.println(9 / 3 - 2); //Affiche 1  
        System.out.println(9 / (3 - 2)); //Affiche 9, c'est mieux :o)  
        //Attention un langage de programmation peut parfois surprendre...  
        System.out.println(10 / 3); //Affiche ??  
    }  
}
```

On obtient 3 seulement. C'est le résultat de la division entière... On verra avec les types comment régler la situation!